

Permutation

置换和排列

置换和排列是各类问题中都很常见的概念。

▼ 本文讨论的不是排列数

本文讨论的主题是全排列，而不是排列组合中的排列数。排列数的相关内容应当参考 [排列组合](#)。

▼ 约定

本文中如果不加说明，总是讨论有限的集合。

定义

一个集合 S 到自身的双射（即一一对应）称为 S 的一个 **置换**（permutation）。如果集合 S 上还具有 [全序](#) 关系，则它的一个置换也常称作一个 **（全）排列**。这个全序关系称为集合上的自然顺序。

► 「置换」与「排列」

设集合 S 的大小是 n ，那么， S 上的全体置换的数目就是 $n!$ 。特别地， $\emptyset$ ，即空集合上有且只有一个置换，也就是空置换。

▼ 记号

置换讨论的是元素间的对应关系，而并不关心元素具体是什么。因而，当讨论大小为 n 的集合时，通常假定讨论的集合就是 $\{1, 2, \dots, n\}$ ；当集合上需要自然顺序的时候，通常假定使用自然数上的自然顺序。

表示方法

置换有多种表示方法。这里，以如下置换为例，讨论不同的置换表示方法。

双行记号

集合 S 上的置换可以表示为

它表示置换 σ 将元素 i 映射到 $\sigma(i)$ 。这里，当然需要 σ 是双射。置换的双行记号表示中，首行的元素出现顺序并不重要，重要的是两行之间的对应关系。

比如说，前文的例子可以按照双行记号写作

当然，也可以写作

单行记号

很多时候，集合 S 上有自然顺序。如果在双行记号中默认首行按照自然的顺序书写，并省略首行，那么，置换可以表示为

这更像自然语言中排列的概念。所以，有时候排列会用来称呼这个有序组。

前文的例子利用单行记号可以写作

这种单行的记号，常用来比较不同排列的大小。

轮换表示

置换还有一种更为紧凑的表达方式，称为置换的轮换表示。它将置换表示为一系列不相交的轮换的乘积。下面描述将给定置换写成轮换表示的步骤。

给定一个置换，可以通过如下步骤写成轮换表示：

1. 如果 S 中还有未曾写下的元素，就写下一个左括号，并写下任意一个这样的元素；
2. 当前一个写下的元素是 i 时，
 - 如果 i 已经在前面写下过，就写上右括号，并返回步骤 1；
 - 如果 i 还没有写下过，就写下 $\sigma(i)$ ，并继续步骤 2；
3. 直到 S 的所有元素都已经写下，结束。

每一对括号中，都是一个轮换。括号中的元素个数，称为对应轮换的长度。实践中，常常省略掉长度为一的轮换。

前文的例子利用轮换表示可以写作

恒等变换中所有的轮换长度都是一，常常记作 (1) 而不是全部省略。

复合

置换的复合就是映射的复合。置换的复合也常常称作置换的乘法。

给定两个置换

那么，它们的乘积 $\sigma\tau$ 的值为

简单来说就是先经过 τ 的映射，再经过 σ 的映射。注意在上面的双行记号中，内层映射 τ 的第二行的顺序和外层映射 σ 的第一行的顺序一致。

因为置换 σ 和 τ 本质是两个映射，所以 $\sigma\tau$ 。置换的复合的运算顺序是自右向左的。置换的乘法并不满足交换律，所以使用错误的顺序计算可能会导致错误的结果。

连续多个置换的乘积称作排列的幂，可以使用 [快速幂](#) 加速计算。

逆置换

因为置换是双射，所以置换总有相应的逆置换。

给定置换

它的逆置换就是

在轮换表示中，只要对每个轮换取逆，就能得到原来的置换的逆；而对每个轮换取逆，只要把元素的书写顺序倒过来就可以了。比如说，上文中的例子 σ 的逆置换的轮换表示是

给定 σ 的一个排列和它的每个元素的排名的序列，就互为逆排列。

轮换

轮换 (cycle) 本身是特殊的置换。轮换的特性是，从轮换中的任何一点 i 出发，都能通过反复应用置换 σ 的方式得到轮换中的另一点 j 。长度为 k 的轮换也称作 **-轮换** (k -cycle)。反复应用 σ^{-1} -轮换 k 次，将得到恒等变换，即每个元素都回到了最开始的位置。

置换的轮换表示可以看作将置换写成这些特殊置换（即轮换）的乘积，因而置换的轮换表示也可以看作是置换的 **轮换分解** (cycle decomposition)。对于每个置换，它分解成轮换乘积的方式在不计顺序后都是唯一的。轮换可以看作是构成置换的基本单元。

置换的轮换分解有着清晰的几何意义。如果将集合 S 上的置换中的每个有序对 $(i, \sigma(i))$ 都看成以 i 为顶点的有向图的边，那么这些轮换就是这个图上面的环路。如果置换 σ 能够写作 k 个置换，就意味着对应的有向图中共计有 k 个环路（包括自环）。这些环路自然互不相交。

不动点

σ -轮换就是置换的 **不动点** (fixed point)。对于集合 S 上的置换 σ ，通常用 $\text{fix}(\sigma)$ 表示 σ 的不动点集合，即 $\text{fix}(\sigma) = \{i \in S \mid \sigma(i) = i\}$ 。

对换

σ -轮换也称作 **对换** (transposition)。也就是说，对换就是只交换了一对元素位置的置换。它的轮换表示是 $(i\ j)$ ，表示它交换了 i 和 j 的位置。

任何置换都可以写作一系列对换的乘积。这相当于说，任何顺序的排列都可以通过一系列交换两个元素的操作恢复成指定的正序排列。这正是基于交换的排序算法在做的事情。

更进一步，[冒泡排序算法](#) 的正确性其实说明，任何置换都可以写作一系列相邻对换的乘积。这里的 **相邻对换** (adjacent transposition) 指的是只交换相邻元素的对换。

性质

在应用中，常常需要关注单个置换的性质。

奇偶性

将轮换分解成对换的方式并不是唯一的。比如，

但是，置换分解成一系列对换时，需要的对换的数目的奇偶性是固定的。一个置换的对换分解的数目的奇偶性也称作置换的 **奇偶性** (parity)。

能够分解成偶数个对换的乘积的置换叫做偶置换，能够分解成奇数个对换的乘积的置换叫做奇置换。当 n 为偶数时，大小为 n 的奇置换和偶置换的数目相同。

符号

根据置换的奇偶性，还可以定义置换的 **符号** (sign)，记作 $\text{sgn}(\sigma)$ 。偶置换的符号定义为 1 ，奇置换的符号定义为 -1 。

置换的乘积的符号，等于它们的符号的乘积，即

也就是说，两个奇偶性相同的置换的复合是偶置换，两个奇偶性不同的置换的复合是奇置换。这一结论，从对换分解的角度看是显然的。

特别地，单次对换必然改变置换的奇偶性。这也正解释了为什么虽然分解成对换的方式不唯一，但是所需的对换的数目的奇偶性是确定的。

置换的符号出现在 [行列式的 Leibniz 展开](#) 中。

置换的阶

置换的阶（order）是指满足如下条件的最小正整数：重复该置换 次后，所有元素都回到了原位。即

有限集合上，所有置换的阶都是有限的。这意味着，从起始顺序出发，只要重复按照固定模式打乱给定序列，在有限时间内，总可以将排列恢复原样。

置换的型

将 个元素的置换做轮换分解，置换的型（cycle type）就是分解中轮换长度的可重集合。这些轮换的长度构成了一个置换长度 的整数分划。如果得到的分解中长度为 的轮换共计 个，那么置换的型常记作

且这些系数满足 。

给定置换的型，不同的置换的数目为

► 分析

如果仅仅给定置换分解成的轮换个数，则不同的置换的数目为 [第一类斯特林数](#)。不同的型的个数为置换长度 的 [分拆数](#)。

从置换的型，可以方便地确定置换的阶和奇偶性等性质。

因为 i -轮换的阶是 i ，不同的轮换又互不相交，所以置换 的阶就是

同样地，因为 i -轮换的奇偶性与 i 的奇偶性相反，所以置换 的奇偶性就是

的奇偶性。这里， c_i 是轮换的个数（包括 1 -轮换，即不动点）。

置换的型在 [Pólya 计数](#) 中有重要作用。

排列相关

如果集合 本身具有自然顺序，此时置换 常称作排列，并用单行记号

表示。注意不要同轮换混淆。

逆序数

在一个排列中，如果某一个较大的数排在某一个较小的数前面，就说这两个数构成一个 **逆序** (inversion) 或反序。这里的比较是在自然顺序下进行的。

在一个排列里出现的逆序的总个数，叫做这个置换的 **逆序数**。排列的逆序数是它恢复成正序序列所需要做相邻对换的最少次数。因而，排列的逆序数的奇偶性和相应的置换的奇偶性一致。这可以作为置换的奇偶性的等价定义。

求解逆序数的算法，可以使用 [归并排序](#) 或 [树状数组](#)，时间复杂度均为 $O(n \log n)$ 。两种算法的解释详见对应章节，这里给出它们的参考实现。

► 参考实现

顺序

排列之间是可以比较大小的。因为每个单行记号就是一个字符串，排列的顺序就是这个字符串上的 [字典序](#)。

在 C++ 的 STL 库 `<ITalgorithm>` 中可以使用 `prev_permutation` 和 `next_permutation` 分别找到当前排列按照字典序的上一个和下一个排列。

排名

将 n 个元素的排列按照字典序从小到大列举出来，则某一排列在这个序列中的位次就是该排列的排名。它建立了排列和正整数之间的一一对应，常常用于排列相关问题的状态压缩。

在中文竞赛圈，这个排名常称作排列的「康托展开」，但这种名称并不规范。更为严谨的说法是，一个排列的排名的康托展开，对应着该排列的 **Lehmer 码** (Lehmer code)。

► 关于「康托展开」

► 示例

从例子中可以知道，求解给定排列的排名的算法，可以分为两步：

1. 将给定的长度为 n 的排列转化为它的 Lehmer 码，即长度为 n 的序列，其中，第 i 位是

也就是在排列中，排在第 i 位后面，但是却比 a_i 小的元素个数。它等于尚未使用的元素中给定元素的排名（减一）。

2. 将 Lehmer 码看作是自然数的康托展开，求出原来的自然数，并加一。也就是说，最终的排名等于

要求解这一问题的逆问题，即给定排名求解相应的排列，只要将上述过程反过来操作即可。在这一过程中求得的 Lehmer 码中的数字之和，就是排列的逆序数。

编程实现时，关键是要能够快速计算「尚未使用的元素中给定元素的排名」（求排名时）和「尚未使用的元素中给定排名的元素」（求排列时），这些都可以通过 [树状数组](#) 或 [线段树](#) 等数据结构维护。正反操作的时间复杂度均为 $O(n \log n)$ 。

► 参考实现

参考资料与注释

- [Permutation - Wikipedia](#)
 - [Lehmer code - Wikipedia](#)
 - [Factorial number system - Wikipedia](#)
-

本页面最近更新：2024/10/25 01:03:52, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[2008verser](#), [aofall](#), [c-forrest](#), [CoelacanthusHex](#), [Early0v0](#), [Great-designer](#), [Marcythm](#), [Persdre](#), [shuzhouliu](#), [Tiphereth-A](#), [Enter-tainer](#), [gavinliu266](#), [gi-b716](#), [hjsjhn](#), [lr1d](#), [MegaOwler](#), [wjy-yy](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用